Documentação técnica

WIN Conecta 50+ · Caderno digital

WIN Business School
05/11/2026

Controle deste documento

Campo	Valor
Versão do documento	1.0.0
Data	05/11/2026
Código a que se refere	commit `ce95c69` (site). A etiqueta `v1.0.0` marca a versão que contém estes documentos (ver `HISTORICO-DE-VERSOES.md`)
Repositório	`acatauassu/conecta`
Branch	`claude/keen-lovelace-47ook9`
Para quem	Time de desenvolvimento da WIN e quem mantiver o projeto
Documentos irmãos	`CHECKLIST-DE-ENTREGA.md`, `MANUAL-DA-METODOLOGIA.md`, `HISTORICO-DE-VERSOES.md`, `qa/README.md`

Sumário

Controle deste documento
1. Visão geral
    1.1 Princípios de arquitetura
2. Estrutura do repositório
    2.1 Ordem de carregamento dos scripts
3. Arquitetura
    3.1 Camadas e fluxo de dados
    3.2 Padrão de uma página de módulo
4. Núcleo JavaScript (`window.WinConecta`)
    4.1 Salvamento automático (`data-salvar`)
    4.2 Componente de pedido (`WinConecta.pedido`)
    4.3 Progresso (`WinConecta.progresso`)
    4.4 Dados do evento (`config.js`)
5. Connect Pack
    5.1 Dados (`connect-pack/connect-pack-data.js`)
    5.2 Navegação
6. Meu caderno
7. Design system
8. Dados guardados no aparelho
9. Acessibilidade
10. Privacidade e segurança
    10.1 Política de segurança de conteúdo (CSP) sugerida
11. Compatibilidade e desempenho
12. Testes e verificação
13. Publicação
14. Como estender
    14.1 Mudar um texto
    14.2 Adicionar um campo que se salva
    14.3 Adicionar um módulo
    14.4 Adicionar ou trocar um modelo do Connect Pack
    14.5 Criar uma nova edição (por exemplo, Universitário)
15. Decisões de arquitetura
16. Limitações conhecidas e riscos
17. Glossário técnico

1. Visão geral

O caderno digital do WIN Conecta 50+ é um **site estático**: HTML, CSS e JavaScript simples, sem servidor, sem banco de dados, sem login e sem etapa de construção (build). Cada participante faz as atividades da noite no próprio celular ou notebook. O que a pessoa escreve fica **no navegador dela** e não é enviado a lugar nenhum.

Item	Valor
Páginas	12: início, 9 módulos, Connect Pack e Meu caderno
Linguagens	HTML, CSS e JavaScript (ES5, sem `let`, `const`, setas ou módulos)
Dependências próprias	Nenhuma. Só a fonte Exo 2, do Google Fonts, que é opcional
Servidor	Qualquer servidor de arquivos estáticos, com HTTPS
Armazenamento	`localStorage` do navegador, com plano B em memória
Tamanho	12 HTML (cerca de 2.800 linhas), 17 JS (cerca de 1.900), 1 CSS (cerca de 270), 5 PNG
Peso por página	50 a 80 KB (HTML, JS e CSS), mais logo e fonte

1.1 Princípios de arquitetura

1. **Estático e sem build.** Quem publica copia a pasta. Quem mantém abre o arquivo e edita.
2. **Privacidade por construção.** Nenhuma chamada de rede do site leva dados da pessoa. O único tráfego externo é a fonte e os links que a própria pessoa abre.
3. **Texto fora do código sempre que possível.** Os textos dos modelos e os dados do evento estão em arquivos de dados.
4. **Acessibilidade como requisito.** Público 50+: letra grande, contraste alto, alvos de toque de 44 px, foco visível, teclado e leitor de tela.
5. **Funciona sem JavaScript até onde dá.** O conteúdo de leitura é HTML. As atividades precisam de JavaScript, e as páginas avisam quando ele está desligado.

2. Estrutura do repositório

```
conecta/
├── site/                       O site (é só isto que se publica)
│   ├── index.html, home.js     Página inicial: módulos e progresso
│   ├── m1/ ... m9/             Um módulo por pasta (index.html + mN.js)
│   ├── connect-pack/           Os 8 modelos (index.html, dados, script)
│   ├── meu-caderno/            Reúne o que a pessoa escreveu
│   ├── assets/
│   │   ├── css/conecta.css     Tokens, componentes, layout e impressão
│   │   ├── js/conecta.js       Núcleo: armazenamento, cartão, copiar, avisos
│   │   ├── js/pedido.js        Componente "pedido pronto para copiar"
│   │   ├── js/progresso.js     Cálculo do progresso por módulo
│   │   ├── js/config.js        Dados do evento (editar aqui)
│   │   └── img/                Logos WIN Conecta (PNG transparente)
│   └── README.md               Resumo para quem publica
├── qa/                         Scripts de verificação (axe, manual, percurso)
├── docs/                       Esta documentação, o manual, o checklist e o histórico
└── (raiz)                      Material de origem: DOCX, PPTX e PDF enviados por Denis
```

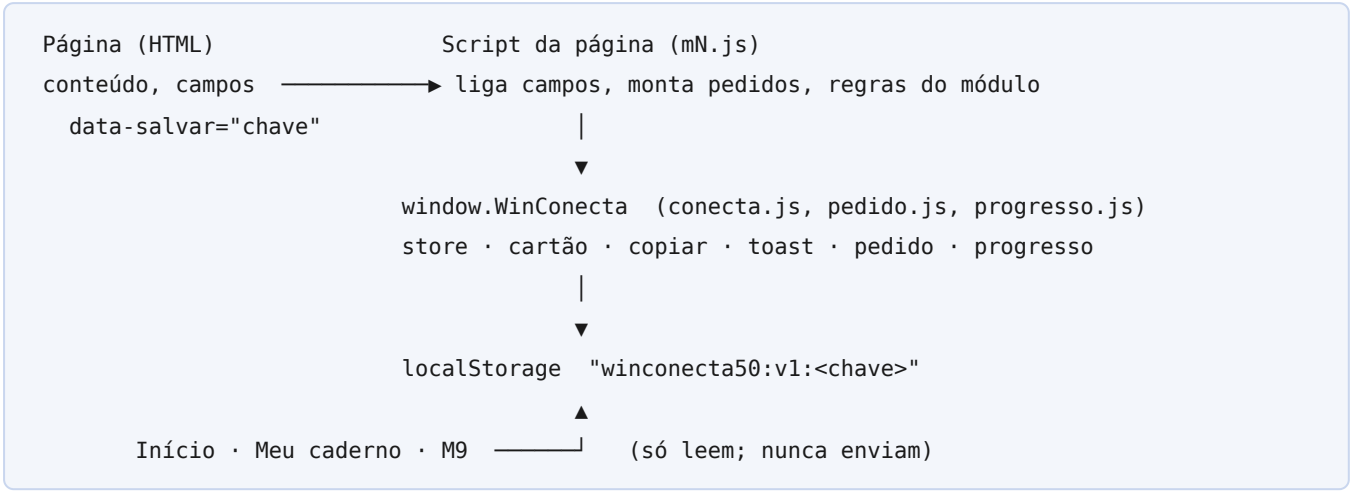
2.1 Ordem de carregamento dos scripts

Todas as páginas de módulo carregam os scripts **nesta ordem**, no fim do `<body>`:

1. `assets/js/config.js` (dados do evento)
2. `assets/js/conecta.js` (núcleo, cria `window.WinConecta`)
3. `assets/js/pedido.js` (só nas páginas que usam pedidos)
4. `assets/js/progresso.js` (só início)
5. `connect-pack/connect-pack-data.js` (só Connect Pack, M9 e Meu caderno)
6. O script da própria página (`mN.js`, `home.js` etc.)

3. Arquitetura

3.1 Camadas e fluxo de dados



Cada página **grava** o que a pessoa escreve. As páginas **Início**, **M9** e **Meu caderno** apenas **leem** esses dados para mostrar progresso, resumo e caderno. Nada sai do aparelho.

3.2 Padrão de uma página de módulo

Todas seguem a mesma estrutura, o que facilita manter e criar novas:

- **Cabeçalho** com o logo pequeno, o link “Início” e “Módulo N de 9”.
- **Abertura** com o gradiente claro e as curvas do folder, o selo, o título (h1) e uma frase.
- **Índice da página** (chips que levam às seções, por âncora).
- **Seções** em fundo alternado (branco e `painel-suave`), cada uma com a duração e um título (h2).
- **Rodapé** com o aviso de privacidade e o link “Apagar tudo que escrevi no curso”.

4. Núcleo JavaScript (`window.WinConecta`)

Definido em `assets/js/conecta.js`. Todos os métodos abaixo estão em `window.WinConecta`.

Item	O que faz
`store.get(chave, padrao)`	Lê um valor do `localStorage` (JSON). Se não existir, devolve `padrao`.
`store.set(chave, valor)`	Grava um valor (JSON). Se o navegador bloquear, grava só em memória.
`store.remove(chave)`	Remove uma chave.
`store.limparTudo()`	Apaga todas as chaves do projeto (só as de prefixo `winconecta50:v1:`).
`CAMPOS_CARTAO`	Lista dos 5 campos do cartão (`quem`, `faco`, `escrevo`, `falo`, `evito`), com rótulo e dica.
`getCartao()`, `setCartao(obj)`	Lê e grava o cartão de contexto.
`cartaoTexto(obj?)`	Junta o cartão em uma frase por campo (“Quem sou: ….”). Vazio se não houver nada.
`CARTAO_EXEMPLO`	O cartão da personagem Marta, usado nos botões de exemplo.
`cartaoEditor(el, {aoMudar})`	Desenha o editor do cartão (5 campos, prévia, copiar, exemplo e apagar).
`toast(mensagem)`	Mostra um aviso curto, anunciado por leitores de tela (`role="status"`).
`copiar(texto, elemento?, msg?)`	Copia para a área de transferência. Se falhar, seleciona o elemento e avisa.
`escapar(texto)`	Escapa HTML. **Use sempre** ao colocar texto da pessoa em `innerHTML`.
`salvarCampos(raiz?)`	Liga o salvamento automático a todo elemento com `data-salvar="chave"`.

4.1 Salvamento automático (`data-salvar`)

Basta colocar o atributo no campo. Funciona com `input` de texto, `textarea`, `select`, caixa de marcar e grupo de opções:

```
<textarea id="p-legenda" data-salvar="m8.legenda"></textarea>
<input type="radio" name="nivel" value="2" data-salvar="entrada.nivel">
```

Em um grupo de opções, **todas** as opções usam a mesma chave. O valor salvo é o `value` da opção marcada. Ao abrir a página, `salvarCampos` restaura o que estava salvo.

4.2 Componente de pedido (`WinConecta.pedido`)

Definido em `assets/js/pedido.js`. Desenha uma caixa com o texto do pedido, os campos para preencher, o botão “Copiar pedido” e o botão “Abrir o ChatGPT”.

```
var p = WinConecta.pedido(document.getElementById('minha-caixa'), {
  chave: 'm8.p2',                // onde salvar os campos (opcional)
  rotulo: 'Pedido 2 · o post',
  template: 'Escolhi a ideia número {{n}}. Escreva um post ...',
  campos: {
    n: { tipo: 'curto', rotulo: 'Número da ideia', marcador: '[n]', exemplo: '3' }
  },
  aoMudar: function (campo, valor) { /* opcional */ },
  rodape: 'texto pequeno abaixo dos botões',
  camposRecolhidos: 'Quer trocar ...?',  // põe os campos dentro de um "abrir"
  chatgpt: true                          // botão "Abrir o ChatGPT" (padrão)
});
p.texto();            // devolve o texto final do pedido
p.definir('n', '5');  // preenche um campo por código
```

Propriedade do campo	Valores e significado
`tipo`	`curto` (uma linha), `longo` (várias linhas), `escolha` (lista, usa `opcoes`) ou `cartao` (vem do cartão de contexto)
`rotulo`, `dica`	Rótulo visível e dica abaixo dele
`marcador`	O que aparece no pedido, em azul, enquanto o campo está vazio
`padrao`	Valor inicial
`exemplo`	Valor usado pelo botão “Ver um exemplo preenchido”. Só aparece o botão se algum campo tiver `exemplo`

Regras do pedido:

- Os `{{campos}}` do texto viram o valor do campo. Texto fora das chaves é copiado exatamente, inclusive colchetes como `[preencher]`.
- Campo vazio aparece como o `marcador`, em destaque, e a caixa avisa quantos faltam. A pessoa pode copiar assim mesmo.
- **Ponto duplicado:** se o valor termina em ponto e o texto do modelo tem outro ponto logo depois, o ponto do modelo é descartado.
- O campo do tipo `cartao` usa `cartaoTexto()` e **não aparece** como campo.

4.3 Progresso (`WinConecta.progresso`)

Definido em `assets/js/progresso.js`. `progresso.modulos()` devolve, para cada módulo: `numero`, `hora`, `titulo`, `pasta`, `feitos`, `total` e `estado`.

- Cada módulo tem uma lista de **sinais** (coisas que a pessoa faz nele). `feitos` conta os sinais cumpridos.
- `estado` é `novo` (nenhum sinal), `andamento` ou `feito` (todos os sinais).
- Um sinal conta como cumprido quando o valor guardado não está vazio. Para grupos (como as 5 caixas da régua), há sinais “todos” e “qualquer”.
- **O progresso não é nota.** Mede se a pessoa preencheu os pontos principais, não se acertou.

Módulo	Sinais (quantos)	O que conta
M1	4	4 passos de acesso; nome; nível; plano
M2	4	algum uso; “o que mudou”; “acertou”; erro ou dúvida
M3	4	situação; os 4 C; cartão; “cartão pronto” (3 caixas)
M4	5	tipo de texto; pedido; algum ajuste; “o que melhorou”; prompt validado
M5	3	pedido; régua inteira (5); “o que encontrei” ou “como ficou”
M6	6	situação; pesquisa; régua e fontes (7); abertura; algum ensaio; “o pedido que funcionou”
M7	3	alguma observação; alguma escolha no jogo; “situação para testar”
M8	6	descrição; 3 ideias; legenda; régua (5); algum dia do calendário; prompt validado
M9	2	modelo escolhido; alguma situação da semana

4.4 Dados do evento (`config.js`)

`window.WIN_CONECTA_CONFIG`, em `assets/js/config.js`. É o arquivo que o time edita no dia.

Campo	Para que serve	Se estiver vazio
`data`, `horario`, `local`	Barra de informações do Início e do M1	Usa os valores já escritos na página
`wifiRede`, `wifiSenha`	Rede e senha, no M1	“A equipe informa na sala.”
`avaliacaoUrl`	Botão da avaliação, no M9	O M9 diz que a equipe passa o link
`proximosCursosUrl`	Botão dos próximos cursos, no M9	O botão não aparece

5. Connect Pack

5.1 Dados (`connect-pack/connect-pack-data.js`)

`window.CONNECT_PACK` é uma lista de 8 modelos. É **aqui** que se edita o texto dos modelos. O arquivo tem instruções no topo.

Campo	Significado
`id`, `titulo`	Número e nome do modelo
`planoTipo`	`gratuito` ou `pago`: define o selo do cartão
`plano`	Texto do aviso de plano, igual ao do caderno impresso
`testadoEm`	Data do último teste, como `"25/11/2026"`. Se for `null`, a data não aparece
`paraQue`, `resultado`, `quando`, `conferir`, `seNaoFuncionar`	Os textos do caderno
`pedido`	O texto enviado à IA, com `{{campos}}`
`campos`	Descrição de cada `{{campo}}` (mesmas propriedades do componente de pedido)

`window.CONNECT_PACK_CARTAO_EXEMPLO` é o cartão de exemplo usado só no botão de exemplo do Connect Pack.

5.2 Navegação

A página usa **âncoras (`#`)**: `#modelo-1` a `#modelo-8` abrem um modelo; `#modelos` mostra a lista; `#cartao` abre o cartão de contexto. A abertura grande só aparece na lista, e, dentro de um modelo, o título do modelo passa a ser o `h1`.

6. Meu caderno

`meu-caderno/caderno.js` monta a página a partir do que a pessoa escreveu, sem enviar nada. O esquema das seções está em uma função (`secoes()`), com um item por campo salvo.

- **Mostra só o que foi preenchido.** Uma caixa permite mostrar também o que está em branco.
- **Imprimir ou salvar em PDF:** usa `window.print()`. O estilo de impressão (`@media print`, em `conecta.css`) esconde menus e botões e deixa tudo em preto e branco.
- **Copiar como texto:** gera um texto simples, com títulos em maiúsculas, para colar no WhatsApp, no e-mail ou nas notas.
- O título vira “Caderno de {nome}” quando a pessoa informou o nome.

7. Design system

Os valores vivem em `assets/css/conecta.css`, no bloco `:root`, e vêm do **WIN Conecta Design Book** (design system mantido na plataforma Claude, privado, versão 5). Os tokens foram medidos no folder digital do evento. Os marcados como “derivado” foram criados para o caderno.

Token	Valor	Uso
`--navy`	`#00144c`	Títulos e texto sobre fundo claro
`--azul`	`#0058fc`	Botão principal, linha de destaque, curvas
`--ciano`	`#04b4e8`	Destaque sobre fundo escuro
`--azul-foto`	`#0034ab`	Borda de foto, estado do botão
`--ceu-alto`, `--ceu-baixo`	`#90bcff`, `#94ccff`	Pontas do gradiente da abertura
`--azul-gelo`	`#eef5ff`	Fundo de apoio (derivado)
`--texto-suave`	`#3b4d7d`	Texto secundário (derivado)
`--borda`	`#5a79bd`	Borda de campos, 4,3:1 sobre branco (derivado)

- **Fonte:** Exo 2, pesos 400, 500, 700 e 800, do Google Fonts, com `"Segoe UI"` e a fonte do sistema como plano B.
- **Temas:** `light` (padrão) e `night`. O tema Noite existe nos tokens (`[data-theme="night"]`) para blocos de fundo navy, e as páginas usam o tema claro.
- **Tamanhos:** texto corrido de 20 px; nenhum texto abaixo de 16 px.

- **Componentes:** 71 classes com prefixo `wc-` (por exemplo `wc-btn`, `wc-painel`, `wc-campo`, `wc-pedido`, `wc-item`, `wc-aviso`, `wc-dica`, `wc-grade`) e 4 do Meu caderno (`cad-`). A lista completa está no CSS.
- **Logos:** `win-conecta-colorido.png` (fundo claro), `win-conecta-branco.png` (fundo escuro), as versões com selo “Edição especial 50+” e a versão pequena do cabeçalho. São PNG transparentes de 2000 px, cortados do arquivo enviado, sem recolorir.

Para mudar uma cor ou tamanho: edite o token no `:root` do `conecta.css`. Se a mudança deve valer também no design system, atualize o `tokens.json` dele, para os dois não divergirem.

8. Dados guardados no aparelho

Prefixo de todas as chaves: **`winconecta50:v1:`**. Valores em JSON.

Trocar o prefixo apaga o que as pessoas já escreveram. Faça isso só em uma versão MAIOR, ou quando duas edições compartilharem o mesmo endereço (use um prefixo por edição). O prefixo está em `conecta.js`, na constante `PREFIXO`.

Chaves comuns

Chave	Valor	O que guarda
`cartao`	objeto `{quem, faco, escrevo, falo, evito}`	Cartão de contexto. Escrito no M3 e no Connect Pack, lido pelos pedidos dos outros módulos
`entrada.nome`	texto	Nome da pessoa (M1). Também título do Meu caderno
`entrada.mesa`	`"1"` a `"6"`	Número da mesa
`entrada.nivel`	`"1"` a `"4"`	Quanto a pessoa usa IA
`entrada.plano`	`gratuito`, `pago` ou `sem-conta`	Plano do ChatGPT
`entrada.quero`	texto	O que quer resolver com a IA
`entrada.dificil`	texto	O que é mais difícil hoje

M1 · Chegada e abertura

Chave	Valor	O que guarda
`m1.acesso.{wifi,chatgpt,app,ola}`	verdadeiro ou falso	Os 4 passos do check de acesso

M2 · Como a IA funciona

Chave	Valor	O que guarda
`m2.usos.{pesquisar,duvidas,escrever}`	verdadeiro ou falso	Como a pessoa já usa a IA
`m2.mudou`	texto	O que mudou entre os dois pedidos
`m2.acertou`	`tudo`, `quase` ou `errou`	Se a IA acertou sobre algo que a pessoa conhece
`m2.errado`	texto	O que estava errado ou inventado
`m2.duvida`	texto	Dúvida para o instrutor ou a mesa
`m2.p2`	objeto `{pessoa, interesse, valor}`	Campos do pedido 2
`m2.p3`	objeto `{assunto}`	Campo do pedido sobre algo que a pessoa conhece

M3 · Os 4 C e o cartão de contexto

Chave	Valor	O que guarda
`m3.situacao`	`convite`, `agradecimento` ou `aviso`	Situação escolhida
`m3.p1`	objeto `{contexto, comando, criterios, conferencia}`	Os 4 C da pessoa
`m3.pronto.{salvei,testei,soou}`	verdadeiro ou falso	Checklist “Meu cartão está pronto?”

M4 · Tarefa 1

Chave	Valor	O que guarda
`m4.tipo`	`email`, `whatsapp`, `proposta`, `convite` ou `redes`	Tipo de texto escolhido
`m4.melhorar`	texto	O que a pessoa queria melhorar
`m4.p`	objeto `{jeito, texto, tom, numero}`	Campos do pedido
`m4.ajuste1`	texto	Primeiro ajuste pedido
`m4.ajuste2`	texto	Segundo ajuste pedido
`m4.melhorou`	texto	O que melhorou e onde a IA errou
`m4.validado`	texto	Meu prompt validado
`m4.diferenca`	texto	O ajuste que fez a diferença
`m4.usar`	texto	Para que vai usar o pedido
`m4.aprendi`	texto	O que aprendeu

M5 · Conferir

Chave	Valor	O que guarda
`m5.p`	objeto `{texto}`	Texto colado no pedido da régua
`m5.regua.{nomes,numeros,fatos,promessas,jeito}`	verdadeiro ou falso	Os 5 itens da Régua
`m5.encontrei`	texto	O que encontrou no texto
`m5.corrigido`	texto	Como ficou depois de corrigir

M6 · Tarefa 2

Chave	Valor	O que guarda
`m6.tipo`	`cliente`, `entrevista` ou `parceiro`	Situação da reunião
`m6.saber`	texto	O que gostaria de saber antes de entrar na sala
`m6.modo`	`pessoa` ou `empresa`	O que quer pesquisar
`m6.p1`	objeto `{nome, cargo, cidade, objetivo}`	Pedido da pesquisa de uma pessoa
`m6.p1e`	objeto `{empresa, cidade, objetivo}`	Pedido da pesquisa só da empresa
`m6.regua.{nomes,numeros,fatos,promessas,jeito}`	verdadeiro ou falso	Régua dos 5 aplicada à reunião
`m6.fonte{1,2}`	verdadeiro ou falso	Abri a primeira e a segunda fonte
`m6.infos`	texto	Informações com fonte que conferiu
`m6.perguntas`	texto	Perguntas que podem fazer
`m6.propor`	texto	O que pode propor
`m6.abertura`	texto	Abertura de 2 minutos
`m6.ensaio.{li,troquei}`	verdadeiro ou falso	Ensaiou em voz alta
`m6.funcionou`	texto	O pedido que funcionou
`m6.util`	texto	A informação mais útil
`m6.ajuste`	texto	O ajuste que fez

M7 · Panorama das outras IAs

Chave	Valor	O que guarda
`m7.obs.{comparei,fonte,link}`	verdadeiro ou falso	O que observou na comparação
`m7.jogo.{a,b,c,d}`	nome da IA	Escolha em cada situação do jogo
`m7.jogo.revelado`	verdadeiro ou falso	Se as respostas do jogo já foram abertas
`m7.testar`	texto	Situação em que vai testar outra IA

M8 · Tarefa 3

Chave	Valor	O que guarda
`m8.descricao`	texto	O que quer divulgar, em 5 linhas
`m8.p1`	objeto `{profissao, faco, quem, oque}`	Pedido das 10 ideias
`m8.ideias`	texto	As 3 ideias escolhidas
`m8.p2`	objeto `{n, linhas}`	Pedido do post
`m8.legenda`	texto	Legenda do post
`m8.chamada`	texto	Chamada do post
`m8.arte`	texto	Ideia da arte
`m8.pub.{promessas,numeros,fatos,imagem,jeito}`	verdadeiro ou falso	Régua antes de publicar
`m8.p3`	objeto `{oque}`	Pedido do calendário
`m8.cal.{1..7}.{c,h}`	texto	Conteúdo (c) e chamada (h) de cada dia do calendário
`m8.p4`	objeto `{n}`	Pedido do roteiro de vídeo (desafio extra)
`m8.validado`	texto	Meu prompt validado
`m8.ajuste`	texto	O ajuste que fez o post soar como a pessoa

M9 · Connect Pack e fechamento

Chave	Valor	O que guarda
`m9.modelo`	`"1"` a `"8"`	Modelo do Connect Pack que vai testar primeiro
`m9.s{1,2,3}`	texto	As 3 situações da próxima semana
`m9.surpreendeu`	texto	O que mais surpreendeu na noite

Connect Pack

Chave	Valor	O que guarda
`pack.m{1..8}`	objeto com os campos do modelo	O que a pessoa preencheu em cada modelo
`pack.c{1..8}`	objeto `{índice: verdadeiro ou falso}`	Itens de “Confira antes de usar” marcados em cada modelo

Pedidos salvos: as chaves `m2.p2`, `m3.p1`, `m4.p`, `m5.p`, `m6.p1` e `m8.p1` a `m8.p4` guardam um objeto com os campos do componente de pedido (nomes dos campos como no código de cada página).

9. Acessibilidade

O site foi desenhado e verificado para o público 50+.

Prática	Como está feita
Idioma	`lang="pt-BR"` em todas as páginas
Título e descrição	Título único por página e `meta description`
Estrutura	Um `h1` por página, sem pular níveis; regiões `header`, `main` e `footer`
Teclado	Link “Ir para o conteúdo” no primeiro Tab; foco visível (contorno de 3 px) em tudo
Campos	Todo campo tem rótulo visível; grupos de opções em `fieldset` com `legend`
Alvos de toque	Pelo menos 44 px (linhas de abrir/fechar, botões de texto e links do cabeçalho incluídos)
Tamanho de texto	Nenhum texto abaixo de 16 px; corrido em 20 px
Contraste	4,5:1 ou mais em todo texto, nos dois temas (verificado pelo axe)
Cor não é o único sinal	Estados dizem a palavra: “Feito”, “Em andamento”, “Fica para depois”
Leitores de tela	`aria-live` nos avisos e no progresso; cronômetros com `role="timer"`
Movimento	`prefers-reduced-motion` desliga a rolagem suave e as animações
Ampliação	Sem rolagem lateral a 320 px; texto com espaçamento aumentado não corta

Como verificar: os scripts de `qa/` (seção 12). **O que falta:** teste com leitor de tela de verdade (VoiceOver e TalkBack) e em aparelhos reais.

10. Privacidade e segurança

- **Nada é enviado.** O site não tem formulário que envie, cookie, análise de uso nem chamada a servidor. O aviso aparece no rodapé de todas as páginas.
- **Tráfego externo:** só `fonts.googleapis.com` e `fonts.gstatic.com` (fonte) e os links que a pessoa abre (ChatGPT, lojas de aplicativos, Perplexity, Gemini), sempre em nova aba e com `rel="noopener noreferrer"`.
- **Texto da pessoa em HTML:** sempre passa por `escapar()` antes de entrar em `innerHTML`. O código não usa `eval` nem `document.write`.
- **Palavra combinada com a família:** o M5 **não tem campo** para ela, de propósito.
- **Apagar:** o link do rodapé chama `store.limparTudo()`. Quem usa outro aparelho ou navegador começa vazio.
- **Se a WIN adicionar análise de uso, cookies ou formulários**, deve primeiro atualizar o aviso de privacidade das páginas.

10.1 Política de segurança de conteúdo (CSP) sugerida

Para servidores que aplicam CSP:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'unsafe-inline';
  style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
  font-src https://fonts.gstatic.com;
  img-src 'self' data:;
  connect-src 'self';
  base-uri 'self'; form-action 'none'; frame-ancestors 'self'
```

Os dois `'unsafe-inline'` existem por dois motivos que podem ser removidos depois: há atributos `style="..."` em 12 páginas, e dois formulários (M1 e M7) usam `onsubmit="return false"`. Para endurecer, mova os estilos para classes do CSS e troque o `onsubmit` por `addEventListener`.

11. Compatibilidade e desempenho

- **JavaScript:** só ES5 (`var` e `function`). Funciona em navegadores antigos.
- **CSS:** usa `grid`, `flex` com `gap`, `clamp()` e `aspect-ratio`. O seletor `:has()` só destaca o item marcado. Em navegador sem ele, perde-se só esse destaque. `text-wrap: balance` é um acabamento opcional.
- **Copiar:** a API `navigator.clipboard` exige **HTTPS**. Em HTTP, o texto fica selecionado para copiar à mão.
- **Aba anônima:** em alguns navegadores o `localStorage` fica indisponível. O site guarda em memória, mas o que a pessoa escreveu se perde ao recarregar. Avise as pessoas.
- **Peso:** 50 a 80 KB por página (HTML, JS e CSS), o CSS tem 4 KB comprimido. O logo do cabeçalho tem 40 KB. O logo da abertura do Início, do M1 e do Connect Pack tem 345 KB.
- **Cache:** durante o evento, use cache curto em JS e CSS para as correções chegarem rápido.

12. Testes e verificação

Os scripts estão em `qa/`, com instruções em `qa/README.md`. Usam Playwright e axe-core, e rodam fora do site (não entram na publicação).

Script	O que verifica
`acessibilidade-axe.js`	axe-core (WCAG 2.1 A e AA e boas práticas) em 13 estados de página, no computador e no celular, nos modos claro e escuro
`acessibilidade-manual.js`	Alvos de toque, texto abaixo de 16 px, títulos, link de pulo, contorno de foco, rolagem lateral a 320 px, texto com espaçamento aumentado
`percurso-completo.js`	Uma pessoa faz do M1 ao M9 no celular, pelos botões “Ir para o Módulo”, e confere o Início e o Meu caderno

Resultado da última execução (05/11/2026): nenhuma violação do axe; todos os alvos com 44 px ou mais (os campos de marcar de 28 px ficam dentro de um rótulo de 56 px); nenhum texto abaixo de 16 px; foco visível em tudo; sem rolagem lateral a 320 px; percurso completo sem falhas e sem erros de console. Também foi conferida a integridade de 117 referências locais (links, imagens e scripts) e a consistência de ids, âncoras e rótulos.

Limites: não houve teste com leitor de tela, em aparelhos reais nem com 30 aparelhos no Wi-Fi do evento. O botão de imprimir não abre a janela na prévia hospedada na plataforma Claude (só no site real).

13. Publicação

1. Copie a pasta `site/` para o servidor, mantendo a estrutura. Os links são relativos e apontam para `index.html` de forma explícita, então funcionam em qualquer caminho.
2. Sirva por **HTTPS**.
3. Edite `assets/js/config.js` com os dados do evento.
4. Conferir `CHECKLIST-DE-ENTREGA.md`: os itens `[BLOQUEIA]` precisam estar marcados antes de abrir o link.
5. Se for embutir em outra página por `iframe`, libere o embutimento no servidor.

O site também roda localmente, para testes:

```
cd site
python3 -m http.server 8000
# abrir http://localhost:8000/
```

Não abra os arquivos com duplo clique: o botão de copiar pode não funcionar fora de `localhost` ou HTTPS.

14. Como estender

14.1 Mudar um texto

- **Texto de um modelo do Connect Pack:** `connect-pack/connect-pack-data.js`.
- **Texto de uma página de módulo:** o `index.html` da pasta. O texto de leitura é HTML puro.
- **Texto de um pedido que a página monta:** o `mN.js` da pasta, no `template` do `WinConecta.pedido`.

14.2 Adicionar um campo que se salva

1. No HTML, coloque `data-salvar="mN.nome"` no campo.
2. Se o campo conta para o progresso, acrescente um sinal no módulo, em `assets/js/progresso.js`.
3. Para aparecer no Meu caderno, acrescente um item na função `secoes()`, em `meu-caderno/caderno.js`.
4. Registre a chave na tabela da seção 8 deste documento.

14.3 Adicionar um módulo

1. Copie a pasta de um módulo parecido e renomeie (por exemplo, `m10/`).
2. Mantenha o cabeçalho, a abertura, o índice, as seções e o rodapé, e a ordem dos scripts (seção 2.1).
3. Acrescente o módulo em `progresso.js` (`MODULOS`) e em `home.js` (`DESCRICAO`).
4. Acrescente a seção no Meu caderno (`caderno.js`).
5. Ligue o módulo anterior e o seguinte pelos botões “Ir para o Módulo”.

6. Acrescente a página à lista `PAGES` dos scripts de `qa/` e rode-os.
7. Atualize o histórico de versões e esta documentação.

14.4 Adicionar ou trocar um modelo do Connect Pack

Edite `connect-pack-data.js`: um objeto por modelo, com o `pedido` e os `campos`. Os cartões do Connect Pack e do M9 leem a lista, e o Meu caderno lê os campos. Se o número de modelos mudar de 8, ajuste também os textos "8 modelos" nas páginas.

14.5 Criar uma nova edição (por exemplo, Universitário)

1. Copie a pasta `site/` para a nova edição.
2. **Troque o prefixo de armazenamento** em `conecta.js` (`PREFIXO`), por exemplo para `winconecta-univ:v1:`, se as duas edições forem ficar no mesmo endereço.
3. Atualize `config.js`, os logos e o texto de cada módulo.
4. Reescreva as tarefas e o Connect Pack para o novo público, seguindo a Parte 2 do manual.
5. Atualize os sinais de progresso e o Meu caderno para os novos campos.
6. Rode os scripts de `qa/` e use o checklist de entrega como modelo.

15. Decisões de arquitetura

Decisão	Por quê	O que se descartou
Site **estático**, sem servidor	O time publica no site da WIN. Menos risco, custo e manutenção	Aplicativo com servidor e banco de dados
O participante usa o **próprio ChatGPT**	Sem custo de API para a WIN, sem enviar dados a ninguém, alinhado ao plano gratuito	IA embutida no site
Dados no `localStorage`, sem login	Privacidade: o maior receio da turma. Nada para vazar	Conta de usuário e sincronização entre aparelhos
JavaScript ES5	Funciona em aparelhos antigos, sem build	Módulos, `let`, `const`, setas
Componente de pedido único	Todos os módulos montam pedidos do mesmo jeito	Pedidos escritos à mão em cada página
Cartão de contexto em 5 campos compartilhados	M4, M6 e M8 colam o cartão sozinhos	Um texto livre por módulo
Progresso **não é nota**	Evitar comparação e cobrança	Placar e pontuação
Sem campo para a palavra combinada	Uma palavra secreta guardada em um aparelho pode ser lida por outra pessoa	Um campo no M5
A avaliação fica **fora do site**	O site não envia dados; um formulário local daria a falsa impressão de envio	Formulário de avaliação no M9
Links entre páginas com `index.html` explícito	Funciona em servidores que não abrem pastas sozinhos	Links para pastas

Decisão	Por quê	O que se descartou
Fonte Exo 2 do **Google Fonts**, com plano B	Fidelidade ao folder; sem ela o site continua legível	Hospedar a fonte (opcional, ver checklist)
Logo pequeno no cabeçalho	O de 2000 px pesava 229 KB em cada página	Usar o logo grande em todo lugar

16. Limitações conhecidas e riscos

Limitação	Efeito	O que fazer
Os dados ficam só no aparelho	Trocar de aparelho ou navegador começa vazio	Avisar na abertura. O Meu caderno permite copiar e imprimir
Aba anônima	O que a pessoa escreveu se perde ao recarregar	Orientar a não usar aba anônima
O site não coleta nada	A WIN não vê o progresso nem a avaliação pelo site	Usar o formulário externo e as folhas de observação
As IAs mudam	Nomes, recursos e limites dos planos podem mudar	Reconfirmar a cada edição e registrar `testadoEm`
Os 8 modelos ainda não foram testados na IA	A data do teste não aparece	Item `[BLOQUEIA]` do checklist
O nome “Gemini Notebook” não foi confirmado	Pode estar desatualizado	Item `[BLOQUEIA]` do checklist
Sem teste em leitor de tela e aparelhos reais	Podem existir problemas não vistos pela verificação automática	Itens T2, T3 e T7 do checklist
`'unsafe-inline'` na CSP	Política menos rígida	Seção 10.1

17. Glossário técnico

Termo	Significado
Estático	Site feito só de arquivos, sem programa rodando no servidor
`localStorage`	Memória do navegador, por endereço, que guarda texto sem prazo
Token	Valor de design com nome (por exemplo, uma cor ou um espaço)
`{{campo}}`	Marcador no texto de um pedido, substituído pelo valor do campo
Âncora (`#`)	Parte do endereço que leva a um trecho da página
CSP	Política que diz de onde a página pode carregar scripts, estilos e fontes
axe-core	Ferramenta que verifica a acessibilidade de uma página
WCAG	Diretrizes internacionais de acessibilidade. O projeto segue o nível AA
Tag (etiqueta)	Marca no repositório que aponta para uma versão exata do código